

Скилл: Модель AnyLogic "СРесурсами1"

Общее описание

Модель симулирует рабочий коллектив. Метод: дискретно-событийное моделирование (СМО).
Язык: Java. Программа: AnyLogic 8.

Основная цепочка (Main)

Source(задания1) → Queue(queue1) → Seize → Delay → Release → Sink

Структура проекта

- **Main** — основная логика
 - **Задание** — класс заявки
 - **Работник** — класс ресурса
 - **Simulation: Main** — эксперимент со слайдерами
 - **База данных** — пока не используется
 - **Ресурсы** — 3D модели, не важны
-

Класс Задание

Параметры:

- `сложность = 35.0`

Переменные:

- `назначенноеВремяДэлайПоДанномуСотруднику` — вычисляется в Seize
 - `выручкаЗаЗадание` — вычисляется в Release
-

Класс Работник

Переменные:

- `здоровье` — начальное triangular(0.3, 1.3, 0.8)
- `продуктивность` — начальное triangular(0.3, 1.3, 0.8)
- `зарплата = 700 * продуктивность` (переменная, не параметр!)
- `больничНачало`, `больничКонец` — моменты времени

- `суммаБольничных` — накопленные расходы на больничные
- `суммаЗарплат` — накопленная зарплата
- `когдаВзятНаРаботу` — момент найма
- `больничныйЧасов` — вспомогательная переменная
- `накопленнаяУсталость` — растёт во время работы, снижается в перерывы/болезнь
- `коэффициентПрезентеизма = 1.0` — снижает продуктивность при выгорании

Переменная `болеет` удалена — была источником race condition. Вместо неё используется `inState(заболел)`.

Диаграмма состояний (`ВзятНаРаботу`):

- `работает` → `заболел` по сообщению `"заболей"` (без доп. условия)
- `заболел` → `работает` по сообщению `"выздоровел"` (без доп. условия)
- `работает` → `перегорает` — пока таймаут 10000 дней (заглушка, переход на условие в планах)
- `перегорает` → `finalState`

При входе в `заболел`:

```
java
больничНачало = time();
```

При выходе из `заболел`:

```
java
больничКонец = time();
```

Событие `СуммированиеЗарплат` (каждый час):

```
java

if (!inState(заболел) && main.расписаниеПерерывов.getValue() == false) {
    суммаЗарплат += зарплата;
    накопленнаяУсталость += 0.003;
    накопленнаяУсталость += Math.max(0, (1.0 - здоровье) * 0.003);
} else {
    накопленнаяУсталость = Math.max(накопленнаяУсталость - 0.0005, 0);
}
```

Два механизма накопления: базовый (трудоголики выгорают при любом здоровье) + от низкого здоровья (без здравоохранения). Примерно половина выгорает по каждому механизму.

Настройки блоков Main

ResourcePool

- Тип: Движущийся
- Количество ресурсов: `количествоРаботников`
- Новый ресурс: Работник
- Перерывы: включены, расписание: `расписаниеПерерывов`
- Может вытеснять (перерыв): ✓
- Аварии/ремонт: включены
 - Время до первой аварии: `Math.max(uniform(0, 365*unit.здоровье), 1)` дней
 - Время между авариями: `Math.max(triangularAV(365*unit.здоровье*0.95, 0.1*unit.здоровье), 1)` дней
 - Время ремонта:
`Math.min(Math.max(triangularAV(2/Math.max(unit.здоровье, 0.01), 0.5/Math.max(unit.здоровье, 0.01)), 0.01), 30.0/7.0)` недель
 - Тип ремонта: Задержка
- При поломке:

```
java
```

```
unit.больничНачало = time();  
send("заболея", unit);
```

- При починке:

```
java
```

```
send("выздоровел", unit);  
unit.больничКонец = time();  
unit.суммаБольничных += (unit.больничКонец - unit.больничНачало) * 24 *  
    unit.зарплата * коэффициентДляБольничных;
```

Seize

- Правило вытеснения задач: Ожидать оригинал ресурса

- Авто приостановка/возобновление: ✓
- При подготовке ресурса:

```
java
```

```
Работник работник = (Работник)unit;
agent.назначенноеВремяДэлэйПоДанномуСотруднику =
    0.105 * agent.сложность / (работник.продуктивность * работник.здоровье);
```

Коэффициент 0.105: без здравоохранения ~33% заданий, с здравоохранением ~100%.

Delay

- Время: `agent.назначенноеВремяДэлэйПоДанномуСотруднику`

Release

- При освобождении ресурса:

```
java
```

```
agent.выручкаЗаЗадание = 3 * agent.назначенноеВремяДэлэйПоДанномуСотруднику * ((Работник)
main.суммаВыручки += agent.выручкаЗаЗадание;
```

Выручка = 3× зарплатных затрат. `суммаВыручки` ≈ 3× `суммаЗарплат`.

Параметры Main

- `количествоРаботников` — слайдер в Simulation
- `коэффициентДляБольничных = 0.75`
- `максБюджетНаЗарплаты = 45000`
- `РаботаетЛиСистемаЗдравоохранения = true` (boolean параметр)

Переменные Main

- `бюджетНаЗарплаты` — динамически = `суммаВыручки * 0.001` (в первые полгода ~25-30 тыс.)
- `суммаЗарплат`, `суммаБольничных` — суммируются из работников каждый час
- `суммаВыручки` — накапливается в Release
- `увольняемый = -1` — вспомогательная (-1 не влияет на популяцию)

События Main

НазначаемИзначальноеЗдоровье

- 1 раз через 1 час

```
java
for (int i = 0; i < работники.size(); i++) {
    работники.get(i).здоровье = triangular(0.3, 1.3, 0.8);
    работники.get(i).продуктивность = triangular(0.3, 1.3, 0.8);
    работники.get(i).когдаВзятНаРаботу = -uniform(0, 365);
}
```

`когдаВзятНаРаботу` отрицательное — разброс стажа 0-365 дней, нет массового увольнения.

ЕстественноеСнижениеЗдоровья

- Каждые 400 часов

```
java
if (!РаботаетЛиСистемаЗдравоохранения) {
    for (int i=0; i < работники.size(); i++)
        работники.get(i).здоровье = Math.max(работники.get(i).здоровье - 0.05, 0.05);
}
```

ПрограммаЗдравоохранения

- Каждые 800 часов, первое в 0

```
java
if (РаботаетЛиСистемаЗдравоохранения) {
    for (int i=0; i < работники.size(); i++)
        работники.get(i).здоровье = Math.min(работники.get(i).здоровье + 0.2, 2.3);
    if (длительностьБолезни > 5) длительностьБолезни -= 5;
}
```

суммаЗарплатИБольничныхСобытие

- Каждый час, первое через 10 часов

java

```
суммаЗарплат = 0;
суммаБольничных = 0;
for(int i=0; i < работники.size(); i++) {
    суммаЗарплат += работники.get(i).суммаЗарплат;
    суммаБольничных += работники.get(i).суммаБольничных;
}
бюджетНаЗарплаты = суммаВыручки * 0.001;
```

увольнение

- Циклическое, каждые 1.5 месяца, первое через 10 недель
- ~3 увольнения за 5 лет при 10 работниках, бюджет 45000

java

```

double бюджетСБуфером = бюджетНаЗарплаты * 0.9;
double максЗарплата = 7500;
double общаяСуммаЗарплат = 0;
double средняяЗарплата = 0;

for (int i = работники.size()-1; i >= 0; i--) {
    Работник р = работники.get(i);
    общаяСуммаЗарплат = 0;
    for (Работник раб : работники) общаяСуммаЗарплат += раб.зарплата;
    средняяЗарплата = общаяСуммаЗарплат / работники.size();

    if ((time() - р.когдаВзятНаРаботу > 545) && (р.продуктивность > 0.7)) {
        double коэффициент = uniform(1.05, 1.15);
        double новаяЗарплата = р.зарплата * коэффициент;
        if ((общаяСуммаЗарплат - р.зарплата + новаяЗарплата <= бюджетСБуфером) &&
            (новаяЗарплата <= максЗарплата)) {
            р.зарплата = новаяЗарплата;
            общаяСуммаЗарплат = 0;
            for (Работник раб : работники) общаяСуммаЗарплат += раб.зарплата;
            средняяЗарплата = общаяСуммаЗарплат / работники.size();
        } else if (новаяЗарплата > максЗарплата) {
            remove_работники(р);
            задержкаНайма.restartTo(time() + uniform(60, 90));
        }
    } else if ((time() - р.когдаВзятНаРаботу > 730) &&
        (р.продуктивность < 0.3) && (р.здоровье < 0.4)) {
        remove_работники(р);
        задержкаНайма.restartTo(time() + uniform(90, 180));
    }
}
}

```

задержкаНайма

- Однократное, начальное время 99999 дней
- Запуск только через `задержкаНайма.restartTo(time() + uniform(60, 90))`

```
java
```

```
add_работники();
```

Расписание перерывов

Пн-пт: 0-9, 13-14, 18-24 = перерыв. Сб-вс: весь день = перерыв. `getValue() == false` = рабочее время. Рабочих часов: $8/\text{день} \times 5 \text{ дней} = 40 \text{ ч/нед}$.

Исправленные баги

1. Здоровье без нижней границы → краш. `Math.max(..., 0.05)`
 2. Событие увольнение оссцuresOnce → cyclic
 3. Race condition с `болеет` → удалена, используется `inState(заболел)`
 4. Единицы суммаБольничных: дни × руб/час → добавлен множитель 24
 5. Таймаут перегорает 320 дней → 10000 дней
 6. когдаВзятНаРаботу = 0 у всех → `-uniform(0, 365)`
 7. задержкаНайма стартовала сама → начальное время 99999
 8. restartTo принимает абсолютное время → `restartTo(time() + uniform(...))`
-

Единицы измерения

- Время модели: **дни**
 - Зарплата: рублей **в час**
 - `больничКонец - больничНачало` → дни → умножать на 24
-

ПЛАН РАБОТ

● Срочно (следующая сессия)

1. Переменная `навык` в Работнике (*первостепенное*) 10% "железных" работников с высоким навыком и медленной деградацией.

```
java
```



```
// В НазначаемИзначальноеЗдоровье:
if (uniform(0,1) < 0.1) {
    работники.get(i).навык = triangular(0.8, 1.5, 1.2); // железный
} else {
    работники.get(i).навык = triangular(0.1, 0.8, 0.4); // обычный
}
```

Навык влияет на:

- скорость деградации здоровья: $\text{здоровье} -= 0.05 / \text{навык}$
- скорость накопления усталости: $\text{накопленнаяУсталость} += 0.003 / \text{Math.max}(\text{навык}, 0.1)$
- скорость выполнения заданий в Seize: делитель добавить $\ast \text{навык}$

2. Переход **работает → перегорает** **по условию** (*первостепенное*) Заменить таймаут 10000 дней на условие $\text{накопленнаяУсталость} \geq 4$. При входе в **перегорает**: $\text{коэффициентПрезентеизма} = 0.5$ Обновить формулу Seize: добавить $\ast \text{работник.коэффициентПрезентеизма}$ в знаменатель.

3. Выход из **перегорает** (*первостепенное*) Добавить состояние **восстанавливается** или переход обратно в **работает** после отдыха. Вариант: через таймаут 30-60 дней → **работает**, $\text{накопленнаяУсталость} = 0$, $\text{коэффициентПрезентеизма} = 1.0$. Либо уход из модели (finalState) если нет поддержки (здравоохранение выключено).

● Малосрочные (ближайшие 2-3 сессии)

4. Выгорание от перенапряжения (сила воли) (*первостепенное*) Добавить в Seize при назначении задания:

```
java
работник.накопленнаяУсталость += Math.max(0, (agent.сложность / работник.продуктивность
```

Смысл: низкопродуктивный работник напрягается сильнее → быстрее выгорает.

5. времяБезРаботника (*первостепенное*) Моделировать период поиска замены после увольнения — очередь растёт, выручка падает. Связать с задержкаНайма: пока нового не наняли, задания накапливаются.

6. частотаБольничных как параметр среды (*вторичное*) Внешний модификатор (сезон, эпидемия) влияющий на **timeBetweenFailures**. Параметр Main: $\text{частотаБольничных} = 1.0$ (множитель).

7. Зависимость зарплаты от навыка (вторичное) Сейчас $\text{зарплата} = 700 * \text{продуктивность}$. Добавить влияние навыка: $\text{зарплата} = \text{базоваяЗарплата} * \text{продуктивность} * (1 + \text{навык} * 0.3)$

● Далёкая перспектива

8. База данных (вторичное) Логировать события (болезни, увольнения, выгорания) для анализа и визуализации.

9. ESG-метрики (вторичное) Добавить переменные для отчётности по GRI 403:

- $\text{коэффициентАбсентеизма} = \text{дни болезни} / \text{рабочие дни}$
- $\text{индексВыгорания} = \text{доля работников в } \text{перегорает}$
- eNPS — зависит от здоровья и зарплаты команды

10. ROI здравоохранения (вторичное) Считать по формуле из статьи: $\text{ROI} = \frac{(\text{экономияБольничных} + \text{приростПродуктивности} + \text{экономияТекучести} - \text{стоимостьПрограммы})}{\text{стоимостьПрограммы}} \times 100\%$

11. Предиктивная модель риска выгорания (далёкая перспектива) Вероятность выгорания работника на горизонте 90 дней по $\text{накопленнаяУсталость}$, здоровье , навык .

12. Кадровый бренд (EVP) (далёкая перспектива) Влияние здравоохранения на скорость найма и качество кандидатов. При $\text{РаботаетЛиСистемаЗдравосохранения}$: уменьшить задержкаНайма и повышать начальные параметры новых работников.

13. NPV программы здравоохранения (далёкая перспектива) Итоговый показатель ценности программы: $\text{NPV} = \text{суммаВыручки(с_здрав)} - \text{суммаВыручки(без_здрав)} - \text{стоимостьПрограммы}$